
EMBEDDED HACKING

Reverse Engineering the Raspberry Pi Pico 2

OPERATION IRON WEB - Requirements & Grading Criteria

ARM Cortex-M33 • GDB • Ghidra

September 27, 2026

For educational and defensive security research only

OPERATION IRON WEB

REQUIREMENTS & GRADING CRITERIA

TARGET: NorthPharma industrial tamper mesh (cabinet intrusion ring)

ARTIFACT: ACT-V.bin / ACT-V.uf2 (compromised)

CREW: FROSTLINE

OPERATIVE: NIGHTINGALE

Project Overview

NorthPharma moves the cabinets that hold what the state does not discuss, and its industrial tamper system built on Pico 2 nodes is the ring of hands on those cabinets. A contractor called **FROSTLINE** planted an implant in the node image: a raw-frame payload handler that matches the **IRONWEB** magic, a mesh propagation loop that re-broadcasts the worm to every peer, a reserved-sector infection marker that re-arms the payload on every boot, and an inverted tamper command authorization verdict. Operative **NIGHTINGALE** recovered the compromised image as **ACT-V.bin**.

Students are the reverse-engineering reserve. They reverse engineer **ACT-V.bin** with Ghidra, find and patch all four defects, defeat the CoreDebug **DHCSR** anti-debug under GDB to observe the marker write, export a corrected image, flash it to a real Pico 2, and prove the corrected behavior on the breadboard. The machine check is `scripts/verify_ctf.py`.

The challenge is a standalone capstone exercise and contains no answer, constant, address, bug, or patch belonging to any other course assignment.

Learning Objectives

- Decode an ARM Cortex-M33 vector and boot table and identify the reset handler and initial stack pointer.
- Map a stripped firmware image into modules by tracing calls from `main` and the monitor loop.
- Locate four corrupted bytes: a raw-frame payload gate, a mesh propagation gate, a reserved-sector marker gate, and an authorization verdict branch.
- Analyze `cbz`, `cbnz`, `beq`, and `bne` condition semantics and branch inversion.
- Explain why a raw pre-authentication payload path is invisible to a sealed protocol and why a worm never needs the cipher.
- Explain why mesh propagation makes one accepted frame a network-wide failure and why a single clean node is not a clean network.
- Explain why reserved-flash state survives a firmware reflash.
- Read CoreDebug **DHCSR**, explain the anti-debug trap, and defeat it under GDB.
- Explain why authentication is not authorization and why a verdict must be verified before the command is applied.

Students must use only the course concepts: ARM registers, stack behavior, USB-CDC and UART consoles, GDB, Ghidra static analysis and binary patching, vector tables, reset startup, XIP, Thumb addressing, condition-code analysis, stateful security, and the Argon2id plus XChaCha20-Poly1305 authenticated envelope.

Deliverables Checklist

#	Deliverable	Format	Criterion
1	Ghidra project screenshot	PNG/JPG	Task 1
2	Vector table and boot table	Inside <code>ACT-V-Answers.md</code>	Task 1
3	<code>main</code> and monitor-loop table	Inside <code>ACT-V-Answers.md</code>	Task 1
4	Module map	Inside <code>ACT-V-Answers.md</code>	Task 1
5	Worm payload evidence and patch	Inside <code>ACT-V-Answers.md</code>	Task 2
6	Propagation gate evidence and patch	Inside <code>ACT-V-Answers.md</code>	Task 3
7	Anti-debug GDB proof, reserved-sector evidence, and patch	Inside <code>ACT-V-Answers.md</code>	Task 4
8	Tamper authorization evidence and patch	Inside <code>ACT-V-Answers.md</code>	Task 5
9	<code>ACT-V_fixed.bin</code>	BIN file	Task 6
10	<code>ACT-V_fixed.uf2</code>	UF2 file	Task 6
11	Hardware proof and reflection	Inside <code>ACT-V-Answers.md</code>	Task 6

Required Tools and Equipment

Tool	Purpose
Raspberry Pi Pico 2	Isolated target node
Debug Probe (OpenOCD)	SWD connection for GDB inspection and the anti-debug work
arm-none-eabi-gdb	Runtime breakpoints, <code>DHCSR</code> clearing, and reserved-sector observation
Ghidra	Static analysis and binary patching
Python 3 with <code>uf2conv.py</code>	UF2 conversion and artifact checks
DHT11, 1602 I2C LCD, RYL998, IR receiver, SG90 servo, 3 LEDs, arm button	Breadboard hardware proof
<code>ACT-V.bin</code> and <code>ACT-V.uf2</code>	Supplied compromised artifacts

Console settings: **USB-CDC virtual COM port, 115200 baud, 8 data bits, no parity, 1 stop bit**. Radio UART settings: **UART1, 115200, network ID 18**.

Artifact Identity

The instructor-issued artifact hashes are:

```

ACT-V.bin          fab47a6e9f81aaccbc01baffb02ae56d98cc70974e82caa256beb6c52bbb4577
ACT-V.uf2          4eac898720e7fc3663a1b5d2b047318230b98a02f7d7a83a0774cc605dc67278
ACT-V_fixed.bin    795fe72e417cfa28bf61196358eb570c18e7f54f655ca90095b4935b1b362fdc
ACT-V_fixed.uf2    fa0809d6a72561e727790981629eb7028006af03f396b875df9e0e720feb36b1

```

The verifier checks the ACT-V.bin and ACT-V_fixed.bin hashes specifically, asserts the four fixed bytes, and requires that only those four offsets differ between the two .bin images. Both .bin images are 51,156 bytes and both .uf2 images are 102,912 bytes.

Grading Rubric - Detailed Breakdown

Task 1: Setup and Initial Analysis (10 points)

Criterion	Points	Full credit	Partial credit	No credit
[DOCUMENT] Ghidra project created with the correct name and settings	2	Project IronWeb_Investigation , raw binary import	One item off	Not set up
[DOCUMENT] Processor configured as ARM Cortex 32 little endian default	2	Screenshot shows the correct processor	Wrong language	Missing
[DOCUMENT] Base address set to 0x10000000	2	Base 0x10000000	Wrong base	Missing
[DOCUMENT] Vector table, initial stack pointer, and reset handler identified	2	Base 0x10000000 , initial SP 0x20082000 , reset handler 0x1000015D	One missing	Not found
[DOCUMENT] main and the tamper controller state machine (monitor_step) addresses identified	1	main 0x10000234 , monitor_step 0x100065F8	One correct	Neither
[DOCUMENT] Module map identifies the latch, control, tamper_auth, implant, and monitor anchors	1	At least one correct anchor per module	Partial	Missing

Task 2: Bug #1 The Worm Payload (20 points)

Criterion	Points	Full credit	Partial credit	No credit
[DOCUMENT] Located the worm payload handler branch at 0x1000A35F	5	Address and function (implant_handle_command) identified	Approximate	Not found
[DOCUMENT] Documented the IRONWEB magic frame and the raw payload gate	5	7-byte IRONWEB magic, 11-byte frame, pre-authentication path	Partial	Wrong
[DOCUMENT & PATCH] Patched 0xD1 to 0xD0 so the IRONWEB frame is ignored	7	Byte 0xD1 changed to 0xD0	Wrong byte	Not patched

Criterion	Points	Full credit	Partial credit	No credit
[DOCUMENT] Explained why the incorrect payload handler turns one frame into an infection	3	Receipt arms the handler and the node infects itself	Vague	Missing

Task 3: Bug #2 The Propagation Gate (20 points)

Criterion	Points	Full credit	Partial credit	No credit
[DOCUMENT] Located the propagation gate branch at 0x1000A2F7	5	Address and function (<code>implant_tick</code> , inlined <code>implant_propagate</code>) identified	Approximate	Not found
[DOCUMENT] Documented the mesh re-broadcast every 4 ticks while infected	5	4-tick interval, <code>IRONWEB</code> frame to peers	Partial	Wrong
[DOCUMENT & PATCH] Patched 0xD1 to 0xD0 so the node does not re-broadcast	7	Byte <code>0xD1</code> changed to <code>0xD0</code>	Wrong byte	Not patched
[DOCUMENT] Explained why a worm needs only one accepted frame to seed the mesh	3	Every peer repeats the frame and becomes a source	Vague	Missing

Task 4: Bug #3 The Infection Marker (20 points)

Criterion	Points	Full credit	Partial credit	No credit
[DOCUMENT] Located the infection marker branch at 0x1000A473	5	Address and inlined <code>implant_init</code> path identified	Approximate	Not found
[DOCUMENT] Documented the CoreDebug DHCSR anti-debug and how it is defeated under GDB	5	<code>0xE000EDF0</code> , <code>C_DEBUGEN</code> and <code>C_HALT</code> , and a real defeat method	Partial	Wrong
[DOCUMENT & PATCH] Patched 0xB9 to 0xB1 so no marker is written to 0x103FF000	7	Byte <code>0xB9</code> changed to <code>0xB1</code>	Wrong byte	Not patched
[DOCUMENT] Explained the reserved sector 0x103FF000 and the write-once marker byte 0xC7	3	Marker, reserved sector, write-once first run	Vague	Missing

Task 5: Bug #4 The Tamper Authorization (20 points)

Criterion	Points	Full credit	Partial credit	No credit
[DOCUMENT] Located the tamper authorization branch at 0x10007569	5	Address and function (<code>control_handle_frame</code>) identified	Approximate	Not found

Criterion	Points	Full credit	Partial credit	No credit
[DOCUMENT] Documented the authorization verdict inversion and the branch condition	5	Reject when the verdict is false	Partial	Wrong
[DOCUMENT & PATCH] Patched 0xB9 to 0xB1 so unauthenticated and replayed commands are rejected	7	Byte 0xB9 changed to 0xB1	Wrong byte	Not patched
[DOCUMENT] Explained that unauthenticated and replayed tamper commands must be rejected	3	The applied command must see only an authorized verdict	Vague	Missing

Task 6: Export and Verify (10 points)

Criterion	Points	Full credit	Partial credit	No credit
[PATCH] Exported ACT-V_fixed.bin from Ghidra	2	Valid patched binary	Corrupt	Not submitted
[PATCH] Converted to ACT-V_fixed.uf2 with the correct base and family	2	--base 0x10000000 --family 0xe48bff59	Wrong flags	Not submitted
[DOCUMENT] scripts/verify_ctf.py passes and hardware proves the correct behavior	3	Verifier passes and the hardware proof is shown	Partial proof	No proof
[DOCUMENT] Reflection maps each of the four defects to a real-world control-system failure	3	Specific mapping for all four	Partial	Missing

Common Pitfalls

Pitfall	Consequence	Avoidance
Reading the payload gate backwards	Receiving the magic still infects the node	Ignore only on the clear-gate branch (beq , 0xD0)
Reading the propagation gate backwards	The node still re-broadcasts the worm	Neutralize only when the gate is clear (beq , 0xD0)
Confusing cbz and cbnz at 0xA473 or 0x7569	The marker is still written, or a failed authorization is still accepted	Neutralize only when the gate or verdict is clear (cbz , 0xB1)
Searching for a standalone implant_infect or implant_propagate symbol	Cannot find the inlined gates	Look inside implant_init at 0x1000A473 and implant_tick at 0x1000A2F7
Patching the shipped image before observing the write	You never prove the marker write	Defeat DHCSR under GDB first, then patch the artifact
Fabricating the GDB session	Verification fails	Show the command sequence and the real observed code path

Pitfall	Consequence	Avoidance
Treating the anti-debug as a defect to patch	Wasted effort; it is identical in both images	Defeat it in a scratch copy or with GDB, then patch the real defect
Missing that the authorization branch is a verdict	Unauthenticated commands still reach the applied command and zone	Accept only when the verdict is true (beq to reject, 0xD0)
Forgetting UF2 conversion	Raw binary will not flash	Use <code>uf2conv.py</code> with family <code>0xe48bff59</code>

How To Breadboard

Device	Pin on device	Pico 2 GPIO	Notes
DHT11 cabinet temperature sensor	DATA	GP4	10 kOhm pull-up to 3.3 V if the module needs it
1602 LCD	SDA	GP2	I2C1, backpack address 0x27
1602 LCD	SCL	GP3	I2C1, 100 kHz
1602 LCD	VCC / GND	VBUS 5 V / GND	The backpack needs 5 V, not 3.3 V
RYLR998	RX	GP8 (Pico TX)	UART1, 115200, network ID 18
RYLR998	TX	GP9 (Pico RX)	UART1
IR receiver	OUT	GP5	VS1838B, internal pull-up enabled
Servo	signal	GP14	PWM 50 Hz; 1000 uF bulk cap across servo 5 V and GND
Red LED	anode	GP16	INTRUSION, 220 to 330 ohm to GND
Yellow LED	anode	GP17	ARMED, 220 to 330 ohm to GND
Green LED	anode	GP18	SECURE, 220 to 330 ohm to GND
Manual arm button	leg 1	GP15	Internal pull-up; leg 2 to GND, never to 3.3 V
Onboard LED	built in	GP25	Heartbeat
Debug Probe	SWCLK / SWDIO / GND	debug header	For GDB only

Use 3.3 V logic on every GPIO. The only 5 V connection is the LCD backpack supply. Keep the 1000 uF capacitor on the servo rail to absorb the SG90 current spike.

Memory Map Reference

Region	Address	Purpose
Bootrom	0x00000000	Immutable boot code
Flash/XIP	0x10000000	Vector table, code, rodata, data image
SRAM	0x20000000	Stack and writable state
CoreDebug_DHCSR	0xE00EDF0	Anti-debug register read by the implant
Implant reserved sector	0x103FF000	Infection marker target (sector)
Implant tick counter	0x200136EC	Incremented once per <code>implant_tick</code>
Implant arming flag	0x20013CEA	Set when the payload handler arms
Implant marker gate	0x20013CEB	Gates the reserved-sector marker write
Implant payload gate	0x20013CEC	Gates the raw-frame payload handler
Implant propagation gate	0x20013CED	Gates the mesh re-broadcast
Implant propagation flag	0x20013CEE	Reports whether propagation is enabled
Tamper command gate	0x20013CE8	Gates the sealed tamper command path
Applied command	0x20013CE7	Applied command after a true verdict
Tamper zone	0x20013CDA	Applied zone after a true verdict
Auth state record	0x200136AC	Anti-replay and state-tag record
Auth field key	0x200136C8	Derived field key for the tag
Latch state	0x20013CEF	Shutter latch state
Latch target	0x20013CF0	Requested latch position

The VA of any file offset is the file offset plus `0x10000000`.

Deadline & Submission

- Create a folder containing the Ghidra screenshot, `ACT-V_fixed.bin`, and `ACT-V_fixed.uf2`.
- Write all written answers in `ACT-V-Answers.md` inside that folder.
- Include the output of `python scripts/verify_ctf.py`.
- ZIP the folder as `lastname-firstname-ACT-V.zip`.
- Submit the ZIP before the posted deadline; late submissions lose 10 percent per day.

Grade Scale

Grade	Percentage	Points
A+	97-100%	97-100
A	93-96%	93-96
A-	90-92%	90-92
B+	87-89%	87-89
B	84-86%	84-86
B-	80-83%	80-83
C	70-79%	70-79
F	0-69%	0-69

Academic Integrity

Use only the supplied Pico 2 and firmware. Do not connect the exercise to an operational industrial control system, a pharmaceutical network, a building-management system, a public network, a military system, or a third-party device. This is a controlled, isolated educational exercise. All analysis and patches must be your own work; sharing binaries, addresses, keys, passphrases, or answers is a violation of the academic integrity policy.

Reference Material

Topic	Reference
ARM Cortex-M33 registers and stack	Course block 1
USB-CDC and UART console capture	Course block 2
Vector tables, reset startup, and XIP	Course block 3
Ghidra static analysis and binary patching	Course block 4
Raw mesh payloads, magic frames, and pre-authentication surface	Course block 5
Mesh propagation, re-broadcast loops, and contamination radius	Course block 6
Reserved-flash persistence and boot re-install	Course block 7
CoreDebug DHCSR and anti-debug	Course block 8
Argon2id and XChaCha20-Poly1305 authenticated envelope	Course block 9